



# Implementação da Visão Computacional: Guia Técnico

A integração da câmera não é apenas "ligar um vídeo", é um fluxo de dados que atravessa três linguagens: **Python** (captura), **JavaScript/Node** (transporte) e **HTML/CSS** (exibição).

## Fase 1: A Captura no Hardware (Python)

**O que acontece:** O robô Cozmo envia dados binários (pixels brutos) via rádio. O Python precisa transformar isso em algo que a web entenda.

- **Função `on_camera_image`:** Criamos um *callback*. Sempre que o sensor da câmera gera um novo frame, esta função é disparada.
  - **Processamento (`Pillow`):** O frame original é pesado. Usamos a biblioteca `Pillow` para converter em **JPEG** com compressão (70%). Isso reduz o uso de banda em quase 10x.
  - **Codificação `Base64`:** Como não podemos enviar arquivos binários facilmente via texto (`stdout`), transformamos a imagem em uma string Base64 (uma sequência de letras e números que representa a imagem).
  - **O Comando `toggle_camera`:** Implementamos uma trava lógica. A câmera só é ativada se o usuário enviar o comando de habilitar, economizando bateria.
- 

## Fase 2: A Ponte de Comunicação (Main & Preload)

**O que acontece:** O Electron atua como o carteiro, levando a imagem do "submundo" do Python para a "superfície" da interface.

- **Main Process (`main.js`):** O Electron escuta o `stdout` do Python. Ao detectar uma mensagem do tipo `camera`, ele a despacha imediatamente para a janela (`mainWindow`) usando o canal `cozmo-log`.
  - **Preload Script (`preload.js`):** Por segurança, a interface não fala direto com o sistema. O Preload cria um túnel seguro (`onCozmoLog`) que permite à interface "ouvir" esses dados sem se expor a riscos.
-

## Fase 3: A Exibição na Interface (Renderer)

**O que acontece:** Transformamos o texto (Base64) de volta em luz na tela do computador.

- **O Componente `<img>`:** No HTML, criamos uma tag de imagem. O segredo é que o atributo `src` dela não aponta para um arquivo no disco, mas recebe a string Base64 dinamicamente: `data:image/jpeg;base64,....`
  - **Lógica de Estado no `home.js`:** 1. O botão começa **desativado**. 2. Ao conectar ao Cozmo, o botão é **liberado**. 3. Ao clicar, enviamos o sinal de "Ligar" e trocamos o texto do botão para "Desligar" (UX).
- 

## Fase 4: Hierarquia de Estados e Segurança

**O Porquê das Travas:**

1. **Ordem de Inicialização:** Impedimos o uso da câmera antes do robô estar pronto para evitar que o código tente ler dados de um processo inexistente (o erro `null`).
  2. **Gerenciamento de Recursos:** Se o aluno fechar o app, o `main.js` envia um `kill` para o Python, garantindo que o robô não fique "preso" tentando transmitir vídeo para ninguém.
- 

## Resumo do Fluxo de Dados

1. **Robô** → Captura o Frame.
2. **Python** → JPEG + Base64 → `print(json)`.
3. **Main** → `stdout.on` → `webContents.send`.
4. **Preload** → `ipcRenderer.on` → `callback`.
5. **Renderer** → `img.src = data`.